Implementation Plan: Circuit Analysis of Tool Trust

Stack: PyTorch, TransformerLens (primary; nnsight fallback for 30B if needed), EAP-IG, standard CKA, SAELens (stretch), ATTC repo (baseline), math benchmarks + `math_verify` checker. Feasible backbone throughout; [STRETCH] = only if time.

Feasibility Gate (do first, blocks everything)

- **Goal:** confirm the models are usable for activation-level work before building anything.
- **Tools:** TransformerLens (`HookedTransformer.from_pretrained`), nnsight fallback.
- **How:**
  - Load ToRL-7B / SimpleTIR-7B / VerlTool-7B (Qwen2.5-Math base) into TransformerLens.
  - Run one problem, confirm `run_with_cache` returns activations and a hook can overwrite one.
  - Confirm the tool-call format has a clean, locatable `<output>...</output>` marker to inject at.
- **Check:** activations read/write works + `<output>` parseable.
- **If fail:** swap model or approach; do not proceed.

Dataset

- **Goal:** matched clean/corrupted tool-conflict pairs on verifiable math.
- **Tools:** vLLM (generation), benchmark loaders (MATH500, GSM8K, Minerva, AIME, AMC), `math_verify`, regex parser.
- **How:**
  - Generate several TIR responses per problem (vLLM).
  - Parse four observables per trace: reasoning answer, tool output, final answer, ground truth.
  - Label accept vs override (Layer 1, natural).
  - Inject correct (clean) vs plausible near-miss wrong (corrupted) tool output at `<output>`, same problem (Layer 2).
  - Split discovery / held-out.
- **Output:** labeled natural set + matched pair set.

- **Check:** clean and corrupted differ only in the injected value; both accept and override cases present.
- **Note:** Layer 2 is load-bearing; parsing the reasoning answer is the hard part (keep only clean-parse traces, else lean on injection).

Metric

- **Goal:** Δ = logit(tool answer) − logit(own answer) at post-output positions.
- **Tools:** TransformerLens (`run_with_cache`, logits), PyTorch.
- **How:**
  - Take final-position logits from a single run.
  - Δ = logit(tool-answer token) − logit(own-answer token).
- **Output:** scalar Δ per run; sign predicts accept/override.
- **Check:** sign matches the observed accept/override label on natural cases.

Circuit Localization

EAP-IG: https://github.com/hannamw/EAP-IG

- **Goal:** find heads/MLPs that causally drive Δ.
- **Tools:** TransformerLens (hooks, `run_with_cache`, `logit_attrs`), EAP-IG, PyTorch (`.backward()`).
- **How:**
  - Cache activations on clean + corrupted runs.
  - Compute Δ, `Δ.backward()` for gradients.
  - Score each component = (act_clean − act_corrupted) × grad(Δ), averaged over IG interpolation steps → rank → shortlist top-k.
  - Activation-patch each shortlisted component (swap clean activation into corrupted run via hook); keep if Δ flips.
  - Label survivors by role: attention pattern (reads tool output?) + DLA (pushes which answer?), grouped by depth.
- **Output:** role-labeled node set = the circuit.
- **Check:** shortlisted patches flip Δ; random components don't.
- **[STRETCH]:** path patching for edges (wiring).

Circuit Validation

- **Goal:** prove the node set is necessary, sufficient, specific.
- **Tools:** TransformerLens (mean/zero ablation hooks), PyTorch.
- **How:**
    - Necessity: ablate circuit -> Delta collapses; compare vs ablating random same-size set; check self-repair (did other components spike?).
    - Sufficiency/faithfulness: mean-ablate everything except circuit -> measure % of Delta recovered.
    - Specificity: intervene -> confirm agreement-case traces + general math accuracy unchanged.
- **Output:** necessity gap, faithfulness %, specificity pass/fail.
- **Check:** circuit-ablation >> random-ablation; faithfulness high (IOI ref ~87%); unrelated behavior intact.

Universality

- **Goal:** does the circuit recur across TIR models?
- **Tools:** CKA (standard implementation); SAELens / crosscoders [STRETCH].
- **How:**
    - Repeat localization on each model.
    - Weak: CKA between models' activations at circuit layers on shared inputs.
    - [STRETCH] Strong: crosscoders / matched components to test shared trust-tool direction across models (address-independent).
- **Output:** CKA scores (weak); shared-feature evidence (strong).
- **Check:** report which holds; expect weak yes, strong partial.

Interventions

- **Goal:** reduce override using the circuit; training-free, token-free.
- **Tools:** TransformerLens (scaling hooks, residual-stream add hooks), PyTorch.
- **How:**
    - Head-scaling: multiply trust-tool heads >1, own-answer heads <1 (via hook at post-output positions).
    - Steering vector: mean(accept activations) - mean(override activations) = trust-tool direction; add to residual stream at post-output positions.
    - Test the two methods separately.
- **Output:** two intervention configs.
- **Check:** each shifts Delta / override behavior at post-output positions only.

Evaluations

- **Goal:** measure the fix vs ATTC.
- **Tools:** ATTC repo (baseline), `math_verify`, held-out set.
- **How:**
    - Per intervention, before vs after: Tool-Ignored rate = overrides / conflict cases; end-task accuracy (checker-verified).
    - Reproduce ATTC as baseline on same models + benchmarks.
    - Report: base / ATTC (with token cost) / ours (zero token cost).
- **Output:** results table.
- **Check:** Tool-Ignored down >=50% relative; accuracy matches ATTC's +4-7.5% at zero added tokens.

Figures

- **Goal:** the visuals that carry the paper.
- **Tools:** matplotlib / seaborn; TikZ or draw.io for the pipeline.
- **List:**
    - (1) Pipeline: problem -> trace -> matched pairs -> EAP-IG -> patching -> validated circuit -> intervention.
    - (2) Circuit map: confirmed components by layer, colored by role.
    - (3) Validation bars: ablate-circuit vs ablate-random vs circuit-only (faithfulness %).
    - (4) CKA heatmap across models.
    - (5) Intervention: Tool-Ignored rate + accuracy, ours vs ATTC vs base (zero-token-cost point marked).
    - [STRETCH] (6) Edge / attribution graph if path patching lands.